GUIDE 24 · AGENTS

Let an AI agent call your saved queries via MCP

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=24-let-an-agent-call-your-queries-via-mcp. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** your saved queries exposed as MCP tools an AI agent can list and call directly - verified here with a real MCP client, not just a curl simulation.

Step 1: Start the MCP server

```shell
pip install "queryapigate[mcp]"
queryapigate mcp
```

```text
Serving MCP tools for /path/to/QUERYAPIGATE_HOME at http://127.0.0.1:5001/mcp
INFO:     Uvicorn running on http://127.0.0.1:5001 (Press CTRL+C to quit)
```

This is a **separate process** from `queryapigate serve` - a different port (`QUERYAPIGATE_MCP_PORT`, default `5001`), speaking Streamable HTTP, not the REST API's WSGI stack. Run both side by side against the same `QUERYAPIGATE_HOME`; they share the same saved queries, connections, keys and audit trail - a run triggered over MCP shows up in the admin UI's History tab exactly like a REST call would.

Step 2: Call it with a real MCP client

Any MCP client that speaks Streamable HTTP and can set a custom request header works. This Python example (the same `mcp` SDK package Claude and other agents use under the hood) is verified end to end against a real running server:

```python
import asyncio
from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

KEY = "your-api-key"

async def main():
    async with streamablehttp_client("http://127.0.0.1:5001/mcp", headers={"X-API-Key": KEY}) as (r, w,
_):
        async with ClientSession(r, w) as session:
            await session.initialize()

            tools = await session.list_tools()
            print([t.name for t in tools.tools])

            result = await session.call_tool("example_top_films", {"top_n": 3})
            print(result.structuredContent)

asyncio.run(main())
```

```text
['example_all_rentals', 'example_film_lookup', ..., 'example_top_films', 'list_tables', 'execute_sql']
isError: False
structuredContent: {'rows': [{'title': 'Electric Signal', 'category': 'Comedy', ...}], 'truncated':
False}
```

The header matters, not the URL. The key travels as an `X-API-Key` request header, the same way every REST request already does - never as a URL query parameter, which would leak it into access logs. Every real MCP client library (this one included) supports setting request headers on the connection; that's the thing to configure, whatever client you're using.

Step 3: Connect a GUI agent (Claude Desktop, or similar)

> **A hosted assistant that signs its user in, rather than taking an API key?** That's OAuth - see Let AI clients sign in through your identity provider.

The mechanics are the same as Step 2 - a Streamable HTTP MCP server at `http://<host>:<port>/mcp`, authenticated with an `X-API-Key` header - but the exact steps to add a *remote* MCP server (as opposed to a local stdio-based one) vary by client and change between versions, so rather than risk giving you stale steps for a UI I can't verify against right now, the reliable move is: look up "add a remote MCP server" / "custom connector" in that specific client's own current documentation, and give it this server's URL (`http://<host>:<port>/mcp`) and the header it needs. If a client's UI doesn't support setting a custom header for a remote server yet, a small local stdio-to-HTTP bridge process can usually stand in - check that client's own current docs for what it recommends, since this space moves quickly and a specific tool name here would likely be stale by the time you read it.

What an agent actually sees

`tools/list` returns one tool per read-only saved query this key can reach - the exact same scoping `GET /catalog` already enforces for REST, so an agent's key never sees or calls more than the identical REST key could - plus two fixed, ad-hoc tools (`list_tables`, `execute_sql`) when the key has any connection-level access at all. See Let an agent explore your schema and run ad-hoc SQL for those two specifically, and Read structured MCP results properly for what `structuredContent` actually contains and why it's shaped the way it is.

A saved query that writes is a tool only when its author marked it for agents (`agent_writes`) and the key's grant allows the write - and then a person approves each write before it runs. See let an AI agent write - with a person approving each write.

Hiding a query from agents while it keeps serving REST. A saved query carries an `mcp` flag (default on). Turn it off and the query is no longer listed or callable as an MCP tool - a call by name is refused with `mcp_disabled` - but it still answers over REST exactly as before. Toggle it in the Console on the API Repository's **MCP** tab (it takes effect at once, no new version), set `"mcp": false` in the query definition when you save it, or `PATCH` `/api/v1/queries/{name}/versions/{version}` with `{"mcp": false}`. Use it for queries that are fine for your apps but you don't want an agent reaching for.

Scoping what an agent can do

Use a real scoped key here, not the admin one - everything from Set up your first scoped API key and Group queries into a collection applies identically to an MCP caller. A key scoped to one collection's worth of read-only reporting queries, with no `connections` grant at all, is a reasonable default for "let an agent explore my data" - it can list and call exactly those queries, nothing ad hoc, and can't reach anything outside that collection even if the model tries.

Check it from the admin UI

Settings > MCP server shows the configured port, an on-demand **Check now** reachability probe (a real TCP connect to that port - never run automatically), and a live **Tools** panel listing exactly what `tools/list` currently returns for an unrestricted caller - useful for confirming what's actually exposed without needing an MCP client at all.

Next steps

- Let an agent explore your schema and run ad-hoc SQL - `list_tables` / `execute_sql`, always read-only.
- Read structured MCP results properly - `outputSchema` / `structuredContent`.
- Set up your first scoped API key - don't hand an agent the admin key.
- Check the MCP server from the Console.
- Let AI clients sign in through your identity provider - OAuth instead of an API key.